

# Lecture 8: Ethernet

EC 441 — Introduction to Computer Networking

Boston University, Spring 2026

## Learning Objectives

By the end of this lecture, you should be able to:

1. Describe the structure of an Ethernet frame and identify the purpose of each field
2. Explain how MAC addresses are structured and allocated (OUI, unicast/multicast/broadcast)
3. Trace an ARP exchange and explain how ARP resolves an IP address to a MAC address
4. Describe how an Ethernet switch self-learns its forwarding table and how it forwards or floods frames
5. Trace frame delivery across a multi-switch LAN
6. Define a subnet and explain its relationship to ARP and routing
7. Identify major Ethernet generations and their speeds, media, and topologies

## 1 Ethernet Overview

Ethernet is the dominant wired LAN technology. It was invented in 1973 by Bob Metcalfe and David Boggs at **Xerox PARC** (Palo Alto Research Center), running at 2.94 Mb/s over a coaxial cable. The name comes from “luminiferous ether,” the hypothetical medium through which light was once thought to propagate.

**Historical note — Xerox PARC.** Ethernet is one of many foundational computing technologies that originated at Xerox PARC in the 1970s. In a single decade, the lab produced the graphical user interface (GUI), the computer mouse, laser printing, object-oriented programming (Smalltalk), the WYSIWYG text editor (Bravo), and the Alto personal workstation — the first computer designed around a GUI and mouse. It also produced Ethernet itself and early work on network protocols. PARC is arguably the most productive industrial research laboratory in the history of computing. Many of these innovations were commercialized not by Xerox but by other companies (most famously Apple and then Microsoft for the GUI), making PARC a recurring case study in the gap between invention and commercialization. The DIX standard (DEC, Intel, Xerox) appeared in 1980 at 10 Mb/s, followed by the IEEE 802.3 standard in 1983. Since then, Ethernet has scaled from 10 Mb/s to 400 Gb/s while maintaining the same basic frame format.

**Why Ethernet won** over competing technologies (Token Ring, FDDI, ATM):

- Simple and inexpensive to implement

- Frame format remained stable through major topology changes
- Open standard enabled competition and drove cost down
- Continuously adapted to new speeds and media

**Topology evolution:** Classic Ethernet placed all hosts on a shared coaxial bus (10BASE-5, 10BASE-2). This was half-duplex; CSMA/CD was essential. The transition to twisted-pair (10BASE-T) introduced a star topology using a central *hub* — which is just a multi-port repeater, still half-duplex and still requiring CSMA/CD. The final step was replacing hubs with *switches*: each host gets a dedicated full-duplex link to the switch, there are no collisions, and CSMA/CD is not needed. The frame format was unchanged throughout all of these transitions.

## 2 Ethernet at the Physical Layer

The frame structure in the next section operates at the link layer. Beneath it, the physical layer converts bits into signals on the wire. Different Ethernet generations have used very different encoding schemes, illustrating the physical-layer tradeoffs from Lectures 3 and 4.

### 2.1 Line Code Requirements

A **line code** maps bit sequences to signal waveforms. A good line code must provide:

- **Clock recovery:** the receiver must synchronize its sampling clock to the transmitter without a separate clock wire. Long runs of identical bits with no transitions cause clock drift.
- **DC balance:** no average DC voltage offset. Ethernet links are typically transformer-coupled (AC-coupled) and cannot pass a DC component.
- **Spectral efficiency:** the signal must fit within the cable or fiber bandwidth. As speeds have increased, cable bandwidth has become the binding constraint.

**NRZ (Non-Return-to-Zero):** the simplest code — 1 = high, 0 = low. Fails clock recovery for long identical runs and has poor spectral efficiency at high bit rates.

### 2.2 Manchester Encoding (10 Mb/s)

All 10 Mb/s Ethernet variants (10BASE-5, 10BASE-2, 10BASE-T) use **Manchester encoding**:

- Bit **1**: low first half, high second half — rising edge at the bit center.
- Bit **0**: high first half, low second half — falling edge at the bit center.

Every bit period contains a mid-bit transition regardless of the data pattern, so clock recovery is always possible. Consecutive identical bits also produce a transition at the *bit boundary* (when one bit ends and the next begins at the same level, the encoding forces a level change to start the new bit correctly). The code is DC-balanced: each bit spends equal time high and low.

**Cost:** the signal must change at twice the bit rate, so the signal bandwidth equals  $2 \times 10 \text{ Mb/s} = 20 \text{ MHz}$ . This  $2 \times$  bandwidth penalty becomes untenable at higher speeds.

**Connection to the preamble:** the 7-byte preamble  $10101010 \times 7$  in Manchester encoding creates a perfectly regular square wave with transitions at every half-bit period — ideal for the receiver’s phase-locked loop (PLL) to lock onto the transmitter’s clock. The SFD byte  $10101011$  ends with two consecutive 1-bits ( $\dots 11$ ), producing a distinctive pattern that cannot occur in the preamble and unambiguously marks “data starts now.”

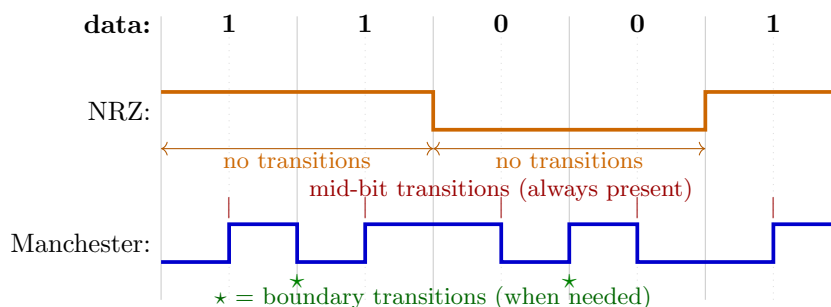


Figure 1: NRZ vs. Manchester encoding for the bit sequence 1 1 0 0 1. NRZ has long flat regions with no transitions (clock recovery failure for runs of identical bits). Manchester guarantees a mid-bit transition on every bit period, enabling clock recovery regardless of the data pattern. Boundary transitions (★) occur between consecutive identical bits to maintain the encoding invariant. The cost is a 2× bandwidth: the Manchester signal for 10 Mb/s Ethernet occupies 20 MHz.

## 2.3 Interframe Gap

After every frame, the transmitter holds the line idle for a mandatory **96-bit period** called the **Interframe Gap (IFG)**. This is how the receiver knows the frame has ended: the line goes quiet. The IFG also gives switch hardware time to finish processing one frame before the next arrives.

## 2.4 Line Codes in Later Ethernet Generations

| Generation | Line code    | Notes                                                                                                   |
|------------|--------------|---------------------------------------------------------------------------------------------------------|
| 10 Mb/s    | Manchester   | Self-clocking; 20 MHz signal bandwidth                                                                  |
| 100 Mb/s   | 4B5B + MLT-3 | 4-bit blocks coded to 5-bit words; MLT-3 cycles through 3 voltage levels to reduce bandwidth to ~31 MHz |
| 1 Gb/s     | 4D-PAM5      | 5-level amplitude modulation on all 4 pairs; trellis FEC                                                |
| 10+ Gb/s   | PAM-4 + DSP  | 4-level PAM with digital signal processing and equalization                                             |

## 2.5 PAM-5 and 1000BASE-T: A Deeper Look

**1000BASE-T** (Gigabit Ethernet over Cat 5e) uses **4D-PAM5** coding. This is one of the most instructive Ethernet line codes: it sits at the intersection of Shannon capacity, modulation theory, cable bandwidth, and forward error correction.

### 2.5.1 Why PAM-5?

Cat 5e cable has a usable bandwidth of approximately 100 MHz. Transmitting 1 Gb/s over a single pair with NRZ would require 500 MHz — far beyond the spec. PAM-5 uses **five voltage levels** (−2, −1, 0, +1, +2), packing more information into each symbol:

$$\log_2(5) \approx 2.32 \text{ bits/symbol}$$

In practice, 2 bits of data are carried per symbol, with the extra capacity used for error correction (see below). At 125 Msymbols/s per pair — well within Cat 5e bandwidth — and with four pairs running simultaneously:

$$4 \text{ pairs} \times 2 \text{ bits/symbol} \times 125 \text{ Msymbols/s} = 1000 \text{ Mb/s}$$

### 2.5.2 The 5th Level and Error Control

Note that  $\log_2(4) = 2$  exactly, so *four* levels would suffice to carry 2 bits/symbol with no redundancy. Using five levels instead of four introduces redundancy — directly analogous to the error-correction ideas from Lectures 5–6: extra capacity at the transmitter is exploited by the receiver to correct errors without retransmission.

#### Key Result

The 5th PAM-5 level is the “extra symbol” that provides FEC redundancy. 1000BASE-T uses this redundancy through a form of **trellis coded modulation** (a convolutional coding technique common in digital communications). The details of trellis coding belong to a digital communications course; what matters here is the principle: using 5 levels instead of 4 provides approximately 5–6 dB of coding gain, at the cost of reduced noise margin per level. The coding gain more than compensates.

### 2.5.3 SNR Tradeoff

More amplitude levels means a smaller gap between adjacent levels for a given peak voltage, making the system more sensitive to noise. This is the fundamental Shannon/Nyquist tradeoff: spectral efficiency vs. noise margin. At 125 Msymbols/s with 5 levels, 1000BASE-T operates close to the Shannon capacity of Cat 5e under typical noise conditions. The trellis coding’s 5–6 dB gain is essential to make the system work reliably.

### 2.5.4 Full-Duplex on All Four Pairs

1000BASE-T runs each pair **full-duplex simultaneously**: both endpoints transmit and receive on the same physical pair at the same time. A **hybrid circuit** separates the transmitted and received signals at each end, and **digital echo cancellation** removes the transmitter’s own signal from what the receiver sees. This is why 1000BASE-T requires Cat 5e (with tighter crosstalk and return loss specifications than Cat 5) rather than the older Cat 5 cable used for 100BASE-TX.

## 2.6 4B5B and MLT-3 (100 Mb/s)

100BASE-TX uses a two-stage line code:

1. **4B5B encoding**: every 4 data bits are mapped to a 5-bit code word selected to guarantee sufficient transitions (no code word contains more than three consecutive zeros). This provides DC balance and clock recovery at the cost of a 25% overhead, increasing the symbol rate from 100 Mb/s to 125 Msymbols/s.
2. **MLT-3 (Multi-Level Transmission, 3 levels)**: the 125 Msymbols/s binary stream is transmitted using three voltage levels (+1, 0, −1). A 1-bit in the 4B5B stream causes the MLT-3 signal to step to the *next* level in the repeating sequence  $+1 \rightarrow 0 \rightarrow -1 \rightarrow 0 \rightarrow +1 \rightarrow$

...; a 0-bit leaves the level unchanged. This cycling reduces the fundamental signal frequency by a factor of 3, bringing the bandwidth to approximately 31 MHz — comfortably within Cat 5's 100 MHz limit.

### 3 Ethernet Frame Structure

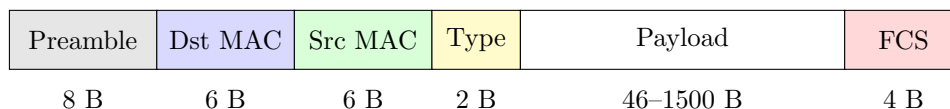


Figure 2: Ethernet frame structure. The preamble and FCS are not passed to higher layers.

#### 3.1 Fields

**Preamble (8 bytes).** The first 7 bytes are 10101010, providing a clock synchronization sequence so the receiver's circuitry can lock onto the sender's bit rate before data arrives. The 8th byte is the *Start Frame Delimiter* (SFD), 10101011; the final two consecutive 1-bits signal that the next bit is the first bit of the destination address.

**Destination MAC Address (6 bytes).** The 48-bit link-layer address of the intended recipient. May be a unicast address, a multicast address, or the broadcast address `FF:FF:FF:FF:FF:FF`.

**Source MAC Address (6 bytes).** The 48-bit address of the sender's interface.

**Type / Length (2 bytes).** This field has a dual interpretation:

- If the value is  $\geq 0x0600$  (1536 decimal): it is an *EtherType*, identifying the encapsulated protocol. Common values: `0x0800` (IPv4), `0x0806` (ARP), `0x86DD` (IPv6).
- If the value is  $\leq 0x05DC$  (1500 decimal): it is a *length* field (the original IEEE 802.3 interpretation), indicating the number of bytes in the payload.

Modern Ethernet almost always uses the *EtherType* interpretation.

**Payload (46–1500 bytes).** The data being carried: typically an IP datagram, an ARP message, or another network-layer packet.

**Frame Check Sequence (FCS, 4 bytes).** A CRC-32 computed over all fields from the destination address through the payload. The receiver recomputes the CRC; if it does not match, the frame is silently discarded. (Ethernet provides no mechanism for retransmission — that is the job of higher layers.)

#### 3.2 Minimum and Maximum Frame Size

##### Key Result

**Maximum frame size:** payload  $\leq 1500$  bytes, giving a maximum frame of  $6 + 6 + 2 + 1500 + 4 = 1518$  bytes. This is the origin of the IP *maximum transmission unit* (MTU) of 1500 bytes.

**Minimum frame size:** payload  $\geq 46$  bytes, giving a minimum frame of 64 bytes.

The minimum frame size is a consequence of CSMA/CD. Recall from Lecture 7: for collision detection to work, the sender must still be transmitting when the echo of a collision returns. With a

10 Mb/s Ethernet and a maximum cable length of 500 m (end-to-end), the round-trip propagation delay is approximately 51.2  $\mu$ s. At 10 Mb/s:

$$L_{\min} = 2\tau R = 51.2 \mu\text{s} \times 10 \text{ Mb/s} = 512 \text{ bits} = 64 \text{ bytes.}$$

If a frame is shorter, the sender finishes transmitting before any collision echo could arrive, so CSMA/CD would fail. If the payload is fewer than 46 bytes, the sender adds *pad bytes* to reach 64 bytes.

In modern full-duplex switched Ethernet, each host has a dedicated link to the switch and collisions cannot occur. The 64-byte minimum is retained for backward compatibility.

**Interframe Gap (IFG).** After every frame, the transmitter holds the line idle for 96 bit-times (9.6  $\mu$ s at 10 Mb/s). This is how the receiver knows the frame has ended: the line goes quiet. The IFG also gives switch hardware time to finish processing one frame before the next arrives.

### Advanced Discussion: Frame Size Distribution in Real Traffic

**TCP ACK padding.** A bare TCP acknowledgement carries no application data: 20 bytes of IP header plus 20 bytes of TCP header totals 40 bytes — 6 bytes short of the 46-byte Ethernet payload minimum. The MAC layer silently appends 6 bytes of padding (typically zeros); the IP **Total Length** field (which still reads 40) tells the receiver where real data ends. This padding is entirely transparent to higher layers. ARP frames, which are only 28 bytes, require 18 bytes of padding.

**Generators of minimum-sized frames.** The following packets are always or frequently padded to the 64-byte minimum:

- TCP pure ACKs, SYNs, FINs, and RSTs (40 bytes each, no payload data)
- ARP requests and replies (28 bytes)
- Small DNS queries (often <46 bytes of UDP payload)
- Interactive traffic with TCP\_NODELAY set (SSH keystrokes, etc.)

**The bimodal distribution.** Empirical measurements of Internet traffic (CAIDA and NLANR packet traces) consistently show that Ethernet frame sizes cluster at two extremes: the 64-byte minimum and the 1500-byte MTU maximum. The distribution is strongly bimodal, with comparatively few frames in between.

**Mice and elephants.** This asymmetry is the “mice and elephants” phenomenon: the majority of all *flows* (and frames by count) are tiny — interactive sessions, DNS lookups, TCP handshakes — yet a small number of bulk-transfer flows near the MTU carry almost all the *bytes*. The lesson is that optimizing for throughput requires efficiently handling the elephants (MTU-sized frames), while optimizing for latency and connection setup time requires handling the mice (minimum-sized frames) with equally low latency.

**Your intuition and the data.** Students often guess that minimum-sized frames account for perhaps 10–20% of traffic by count. The measured figure — closer to 40–50% — is reliably surprising. Note also that the fraction by bytes is below 5%: the same data set that looks “dominated by tiny frames” when plotted by count looks “dominated by large frames” when plotted by bytes. Both views are correct; they describe fundamentally different things.

**What the NIC strips vs. what the OS sees.** A common question: does the NIC strip the padding before handing the frame to the operating system? No — padding bytes are delivered to the OS. Here is the precise breakdown:

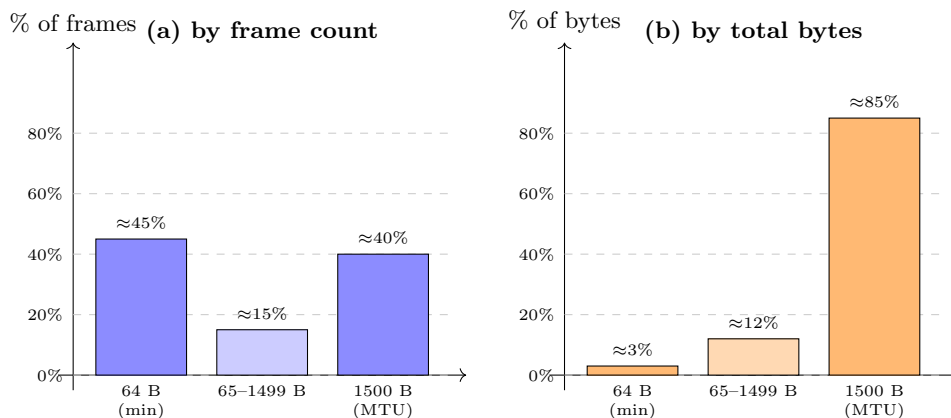


Figure 3: Schematic bimodal distribution of Ethernet frame sizes, representative of empirical Internet traffic measurements (CAIDA, NLNR traces). Numbers are approximate; the exact split varies by vantage point and traffic mix. **(a)** Nearly half of all frames by *count* are minimum-sized (64 bytes): pure TCP ACKs/SYNs, ARP messages, DNS queries, and other small control traffic. **(b)** By total *bytes*, minimum-sized frames are negligible; nearly all bytes are carried in frames near the 1500-byte MTU. This contrast illustrates the “mice and elephants” phenomenon.

- **NIC strips:** preamble, SFD, and FCS. The FCS is verified in hardware; frames that fail CRC are silently dropped before the OS ever sees them.
- **NIC delivers to the OS** (via DMA into a host memory ring buffer): destination MAC through the last byte of the payload, including any padding. A 40-byte TCP ACK in a minimum frame arrives in memory as 60 bytes (14-byte Ethernet header + 46 bytes of padded payload).
- **The IP stack ignores the padding** by reading the **Total Length** field in the IP header (value 40 for a bare ACK) and processing exactly that many bytes. TCP never sees the trailing pad bytes. Transparency to upper layers comes from the IP length field, not from the NIC stripping anything.

Modern NICs with hardware offload engines (checksum verification, Large Receive Offload, Receive Side Scaling) do parse IP and TCP headers — but even these do not strip Ethernet padding. Wireshark captures on a live wired interface will show padded frames and correctly label the extra bytes as “Ethernet padding” while the IP dissector decodes only the bytes indicated by **Total Length** — a useful classroom illustration of this mechanism.

**Practical implications.** The bimodal distribution shapes real switch and router design:

- At 10 Gb/s, minimum-sized frames arrive at approximately  $10^{10}/(64 \times 8) \approx 19.5$  million frames per second. Forwarding table lookup must keep pace even for tiny frames.
- Buffer memory sizing must accommodate bursts of large (near-MTU) frames.
- In software-based forwarding (e.g., a Linux router), interrupt and CPU overhead scales with frame *count*, not byte count. This is a primary motivation for hardware-accelerated forwarding (NIC offload, DPDK, or dedicated ASICs) even when aggregate throughput appears manageable.

## 4 MAC Addresses

A **MAC address** (also called an Ethernet address, hardware address, or burned-in address) is a 48-bit identifier assigned to a network interface. It is written in hexadecimal with colon (or hyphen) separators: `A4:C3:F0:85:AC:2D`.

### 4.1 Structure

| Bytes | Field        | Description                                                           |
|-------|--------------|-----------------------------------------------------------------------|
| 1–3   | OUI          | Organizationally Unique Identifier; assigned to manufacturers by IEEE |
| 4–6   | NIC-specific | Assigned by the manufacturer; unique within the OUI                   |

Two special bits reside in the first byte:

- **I/G bit** (least significant bit of byte 1): 0 = unicast (addressed to one interface), 1 = multicast (addressed to a group of interfaces).
- **U/L bit** (second least significant bit of byte 1): 0 = globally administered (factory-set), 1 = locally administered (overridden by software).

The broadcast address `FF:FF:FF:FF:FF:FF` (all bits set) is a special case of multicast; every interface on the LAN receives and processes frames addressed to it.

### 4.2 Flat vs. Hierarchical Addressing

MAC addresses are **flat**: they carry no information about where on the network the device is located. This is appropriate for hardware identifiers but makes routing at scale impossible.

IP addresses are **hierarchical**: the network prefix encodes the device's location in the routing topology, enabling routers to aggregate routes. A device that moves between subnets acquires a new IP address but retains its MAC address.

The distinction drives the need for ARP: the network layer knows the IP destination, but the link layer needs a MAC address to build the frame.

An important property of Ethernet: **the MAC addresses in a frame change at each hop**. When a router forwards a datagram between subnets, it strips the old Ethernet frame and builds a new one with fresh source and destination MAC addresses for the outgoing link. The IP source and destination addresses remain unchanged end-to-end.

## 5 ARP — Address Resolution Protocol

### 5.1 The Problem

Suppose Host A (IP: `192.168.1.10`, MAC: `AA:AA:AA:AA:AA:AA`) wants to send an IP datagram to Host B (IP: `192.168.1.20`) on the same subnet. Host A must construct an Ethernet frame with a destination MAC address, but it does not know Host B's MAC address. ARP solves this.



## 5.2 ARP Exchange

### ARP for a same-subnet destination

#### Step 1 — ARP Request (broadcast):

- Ethernet dst: FF:FF:FF:FF:FF:FF (broadcast; all hosts on LAN receive this frame)
- Content: “I am 192.168.1.10 (AA:AA:AA:AA:AA:AA). Who has 192.168.1.20? Tell me your MAC.”

#### Step 2 — ARP Reply (unicast):

- Ethernet dst: AA:AA:AA:AA:AA:AA (back to Host A only)
- Content: “I am 192.168.1.20; my MAC is BB:BB:BB:BB:BB:BB.”

#### Step 3 — Datagram sent:

Host A builds the Ethernet frame: dst MAC = BB:BB:BB:BB:BB:BB, src MAC = AA:AA:AA:AA:AA:AA.

Note that the request is broadcast but the reply is unicast. This asymmetry reduces unnecessary traffic: only the requester needs the answer.

## 5.3 ARP Table

Each host maintains an **ARP table** (ARP cache), which maps IP addresses to MAC addresses:

| IP Address   | MAC Address       | TTL   |
|--------------|-------------------|-------|
| 192.168.1.20 | BB:BB:BB:BB:BB:BB | 18:32 |
| 192.168.1.1  | CC:CC:CC:CC:CC:CC | 19:55 |

Entries have a TTL (typically ~20 minutes). When Host A needs to reach an IP address and the entry is not in the table (cache miss), it sends an ARP request first. This adds one round-trip of latency to the first datagram, but subsequent datagrams to the same host use the cached entry.

Hosts can also learn from ARP requests they overhear: the request contains the sender’s IP and MAC, so every host on the LAN can update its cache even if it was not the target.

## 5.4 ARP for Off-Subnet Destinations

ARP operates **within a single subnet** (broadcast domain). If Host A needs to reach an IP address on a different subnet, it cannot ARP for that address — the ARP broadcast would not cross the router.

Instead:

1. Host A determines (from its routing table) that the next hop is the **default gateway**, e.g., 192.168.1.1.
2. Host A ARPs for the gateway’s MAC address.
3. The Ethernet frame carries the IP datagram to the router; the IP destination is still the original remote host.

4. The router strips the Ethernet header, looks up the next hop, and sends a new frame on the outgoing interface.

This is the key mechanism that separates Ethernet (link-layer) addressing from IP (network-layer) addressing: IP handles end-to-end routing; Ethernet handles one-hop delivery.

## 5.5 Gratuitous ARP

A host may broadcast an ARP reply in which the sender IP and target IP are the same (its own address). This is called **gratuitous ARP**. Uses include:

- Duplicate address detection: if another host replies, there is an IP conflict.
- Cache refresh: forcing other hosts to update their ARP tables (e.g., after a failover in a high-availability cluster).

## 5.6 ARP and IPv6

ARP is specific to IPv4. IPv6 uses the **Neighbor Discovery Protocol** (NDP), which serves the same function using ICMPv6 messages (Neighbor Solicitation and Neighbor Advertisement). NDP uses multicast rather than broadcast, which reduces unnecessary traffic.

# 6 Ethernet Switches

## 6.1 Hub vs. Switch

A **hub** is a multi-port repeater: it receives a signal on one port and retransmits it on all other ports. All devices share a single collision domain. Hubs are half-duplex; CSMA/CD governs access.

A **switch** forwards frames selectively based on destination MAC address. Each port is its own collision domain; with full-duplex point-to-point links, no collisions occur at all. The switch can forward multiple frames simultaneously on independent port pairs.

## 6.2 The Forwarding Table

A switch maintains a **forwarding table** (sometimes called the MAC address table or CAM table), which maps MAC addresses to port numbers:

| MAC Address       | Port | TTL  |
|-------------------|------|------|
| AA:AA:AA:AA:AA:AA | 1    | 4:58 |
| BB:BB:BB:BB:BB:BB | 3    | 4:45 |
| CC:CC:CC:CC:CC:CC | 2    | 4:12 |

**Forwarding rule:** when a frame arrives,

1. Look up the destination MAC in the forwarding table.
2. If the entry is found: **forward** the frame out the indicated port only.
3. If the entry is not found: **flood** — forward out all ports except the ingress port. This is also called *unknown unicast flooding*.
4. If the destination is a broadcast or multicast address: flood all ports except ingress.

### 6.3 Self-Learning

The forwarding table is built automatically — no administrator configuration is needed:

#### Key Result

**Learning rule:** when a frame arrives on port  $P$  with source MAC address  $X$ , record the mapping ( $X \rightarrow P$ ) in the forwarding table with a fresh TTL.

#### Self-learning trace (three hosts on ports 1, 2, 3; table empty)

1. A sends to B (arrives on port 1): learn  $A \rightarrow$  port 1. B unknown  $\rightarrow$  flood ports 2 and 3.
2. B sends to A (arrives on port 2): learn  $B \rightarrow$  port 2. A known (port 1)  $\rightarrow$  forward to port 1 only.
3. C sends to B (arrives on port 3): learn  $C \rightarrow$  port 3. B known (port 2)  $\rightarrow$  forward to port 2 only.

After these three exchanges, the switch has learned all host locations.

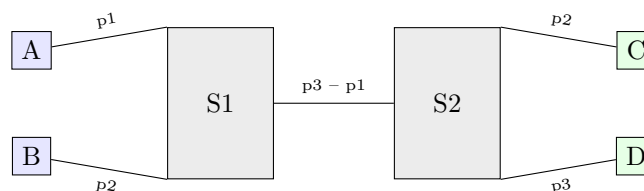
Entries expire after an inactivity timeout (typically  $\sim 300$  seconds). When a host is moved to a different port, the stale entry ages out and the switch relearns the host's location from the next frame it sends.

### 6.4 Simultaneous Forwarding

A modern switch can forward multiple frames at the same time as long as they use different ports. For example,  $A \rightarrow B$  on ports  $1 \rightarrow 2$  and  $C \rightarrow D$  on ports  $3 \rightarrow 4$  can proceed simultaneously with no interference. This is why switch performance is rated in aggregate throughput, not per-port.

## 7 Multi-Switch Example

Consider the topology below: hosts A and B connect to switch S1; switch S1 connects to switch S2 via a trunk link; hosts C and D connect to S2.



**Scenario: A sends to D (both tables empty).**

1. Frame arrives at S1 on port 1, src=A, dst=D. S1 learns  $A \rightarrow p1$ . D unknown  $\rightarrow$  S1 floods p2 (to B) and p3 (to S2). B receives but discards (not for B).
2. Frame arrives at S2 on port 1, src=A, dst=D. S2 learns  $A \rightarrow p1$ . D unknown  $\rightarrow$  S2 floods p2 (to C) and p3 (to D). C receives but discards; D accepts.

**Scenario: D replies to A.**

1. Frame arrives at S2 on port 3, src=D, dst=A. S2 learns  $D \rightarrow p3$ . A is known  $(p1) \rightarrow S2$  forwards to p1 only (toward S1).
2. Frame arrives at S1 on port 3, src=D, dst=A. S1 learns  $D \rightarrow p3$ . A is known  $(p1) \rightarrow S1$  forwards to p1 only. A receives.

After these two exchanges, both switches have learned the locations of A and D. All subsequent  $A \leftrightarrow D$  traffic is forwarded without flooding.

**Key properties of switches:**

- **Transparent:** end hosts do not know switches exist; they see no change in how they address frames.
- **Plug-and-play:** self-learning requires no configuration.
- **Loop avoidance:** multiple switches connected in a loop cause broadcast storms. The **Spanning Tree Protocol** (STP, IEEE 802.1D) identifies and blocks redundant links to ensure the active topology is loop-free. STP is mentioned here as a concept; detailed treatment is beyond this course.

## 8 Subnets

A **subnet** (subnetwork) is a set of IP interfaces that share the same network prefix and can communicate at the link layer without going through a router.

In CIDR notation,  $192.168.1.0/24$  means the first 24 bits are the network identifier; the remaining 8 bits identify individual hosts (addresses  $192.168.1.0$  through  $192.168.1.255$ ). A router interface, a host, or any other IP interface belongs to a subnet if its address matches the prefix.

**Subnets and Ethernet:**

- Each subnet corresponds to a single Ethernet broadcast domain. An ARP broadcast sent by any host on the subnet reaches every other host on the subnet.
- A switch connects hosts within a subnet; it does not require or examine IP addresses.
- A router connects subnets; each router interface belongs to a different subnet. The router strips and rewrites Ethernet headers as it forwards datagrams between subnets.

## 9 Ethernet Technology Survey

IEEE 802.3 media designations follow the convention **<speed>BASE<medium>**: the speed is in Mb/s (for 10/100/1000) or Gb/s (for 10G and above); BASE indicates baseband signaling; the suffix identifies the medium or approximate distance.

## 9.1 Historical Technologies

| Standard            | Year | Speed   | Medium              | Topology               |
|---------------------|------|---------|---------------------|------------------------|
| 10BASE-5 “Thicknet” | 1980 | 10 Mb/s | RG-8 coaxial cable  | Bus (vampire taps)     |
| 10BASE-2 “Thinnet”  | 1985 | 10 Mb/s | RG-58 coaxial cable | Bus (BNC T-connectors) |
| 10BASE-T            | 1990 | 10 Mb/s | Cat 3 UTP           | Star with hub          |

Historically, coaxial-bus Ethernet and hub-based Ethernet were the norm — switched Ethernet now dominates.

## 9.2 Modern Technologies

| Standard         | Year | Speed    | Medium               | Typical Use             |
|------------------|------|----------|----------------------|-------------------------|
| 100BASE-TX       | 1995 | 100 Mb/s | Cat 5 UTP (2 pairs)  | Desktop/campus          |
| 1000BASE-T       | 1999 | 1 Gb/s   | Cat 5e UTP (4 pairs) | Desktop/campus standard |
| 1000BASE-SX/LX   | 1998 | 1 Gb/s   | MM/SM fiber          | Data center             |
| 10GBASE-T        | 2006 | 10 Gb/s  | Cat 6A UTP           | Server uplinks          |
| 10GBASE-SR/LR    | 2002 | 10 Gb/s  | MM/SM fiber          | Data center             |
| 25GBASE-SR       | 2016 | 25 Gb/s  | Multimode fiber      | Server-to-ToR switch    |
| 40GBASE-SR4      | 2010 | 40 Gb/s  | Multimode fiber      | Aggregation             |
| 100GBASE-SR4/LR4 | 2010 | 100 Gb/s | Fiber                | Spine/backbone          |
| 400GBASE-SR8     | 2017 | 400 Gb/s | Fiber                | Hyperscale data centers |

## 9.3 Power over Ethernet

IEEE 802.3af/at/bt defines standards for delivering DC power over the same twisted-pair cable as data. The power sourcing equipment (PSE), typically a switch, provides current that the powered device (PD) converts to its operating voltage. Power levels: 802.3af (15.4 W), 802.3at (30 W), 802.3bt (up to 90 W). Common applications include IP phones, wireless access points, IP cameras, and IoT sensors.

## 9.4 Auto-Negotiation and Duplex Mismatch

Ethernet interfaces can **auto-negotiate** speed and duplex: both ends exchange capability advertisements and select the highest mutually supported mode. A **duplex mismatch** — one end full-duplex, the other half-duplex — is a common misconfiguration. The full-duplex end transmits without backing off; the half-duplex end interprets simultaneous traffic as collisions. Symptoms include severely degraded throughput and high CRC error counts on the half-duplex interface.

### Late Collisions: the Diagnostic Signature of a Duplex Mismatch

A **late collision** is a collision detected *after* the first 512 bits (64 bytes) of a frame have been transmitted. Under a correctly configured network this should be impossible: the entire point of the 64-byte minimum frame size is that a sender is still transmitting when any collision echo arrives. Detecting a collision after byte 64 means that invariant was violated.

#### How a duplex mismatch produces late collisions:

- The full-duplex end transmits whenever it has data — no CSMA/CD, no carrier sense.

- The half-duplex end follows CSMA/CD: it checks that the wire is idle, starts transmitting, and listens for collisions during the slot time.
- If the full-duplex end starts transmitting *after* the half-duplex end has already passed byte 64, the half-duplex end detects a signal collision — but too late. That is a late collision.

#### What each side reports:

| Side        | Symptom                                                                 |
|-------------|-------------------------------------------------------------------------|
| Half-duplex | Late collision counter increments; frame retransmitted (up to 16 times) |
| Full-duplex | FCS/CRC errors (half-duplex side jammed the line mid-frame)             |

Both sides see degraded throughput, yet neither interface goes down — the link stays up and appears healthy. This is what makes the mismatch so insidious: it is invisible without explicitly checking the duplex configuration or inspecting interface error counters.

**Cisco show interface** is the classic diagnostic: a late collision counter incrementing on a half-duplex interface is the canonical symptom of duplex mismatch. A port that failed auto-negotiation and fell back to half-duplex while the other end was hard-coded full-duplex produces this signature reliably.

**Other causes of late collisions** (less common in modern switched networks):

- Cable exceeding the maximum segment length — propagation delay too large for the 512-bit slot time.
- Faulty cable with signal reflection or excessive attenuation.
- Too many repeater hops in a legacy coaxial or hub-based network.

## References

- Wikipedia contributors. “PARC (company).” Wikipedia. [https://en.wikipedia.org/wiki/PARC\\_\(company\)](https://en.wikipedia.org/wiki/PARC_(company))
- Wikipedia contributors. “Ethernet.” Wikipedia. <https://en.wikipedia.org/wiki/Ethernet>
- Wikipedia contributors. “Ethernet frame.” Wikipedia. [https://en.wikipedia.org/wiki/Ethernet\\_frame](https://en.wikipedia.org/wiki/Ethernet_frame)
- Wikipedia contributors. “MAC address.” Wikipedia. [https://en.wikipedia.org/wiki/MAC\\_address](https://en.wikipedia.org/wiki/MAC_address)
- Wikipedia contributors. “Address Resolution Protocol.” Wikipedia. [https://en.wikipedia.org/wiki/Address\\_Resolution\\_Protocol](https://en.wikipedia.org/wiki/Address_Resolution_Protocol)
- Wikipedia contributors. “Network switch.” Wikipedia. [https://en.wikipedia.org/wiki/Network\\_switch](https://en.wikipedia.org/wiki/Network_switch)
- Wikipedia contributors. “Spanning Tree Protocol.” Wikipedia. [https://en.wikipedia.org/wiki/Spanning\\_Tree\\_Protocol](https://en.wikipedia.org/wiki/Spanning_Tree_Protocol)
- Wikipedia contributors. “Subnetwork.” Wikipedia. <https://en.wikipedia.org/wiki/Subnetwork>

- Wikipedia contributors. “IEEE 802.3.” Wikipedia. [https://en.wikipedia.org/wiki/IEEE\\_802.3](https://en.wikipedia.org/wiki/IEEE_802.3)
- Wikipedia contributors. “Power over Ethernet.” Wikipedia. [https://en.wikipedia.org/wiki/Power\\_over\\_Ethernet](https://en.wikipedia.org/wiki/Power_over_Ethernet)
- Wikipedia contributors. “Manchester code.” Wikipedia. [https://en.wikipedia.org/wiki/Manchester\\_code](https://en.wikipedia.org/wiki/Manchester_code)
- Wikipedia contributors. “1000BASE-T.” Wikipedia. <https://en.wikipedia.org/wiki/1000BASE-T>
- Wikipedia contributors. “Trellis coded modulation.” Wikipedia. [https://en.wikipedia.org/wiki/Trellis\\_coded\\_modulation](https://en.wikipedia.org/wiki/Trellis_coded_modulation)
- Wikipedia contributors. “Traffic generation model.” Wikipedia. [https://en.wikipedia.org/wiki/Traffic\\_generation\\_model](https://en.wikipedia.org/wiki/Traffic_generation_model)
- CAIDA. “CAIDA Internet Traffic Measurement Data.” <https://www.caida.org/data/>